

Contents

1	Math	
1.1	Fpow	1
1.2	Combination	1
1.3	Extgcd	1
1.4	質數	1
1.5	模反元素	1
2	Shortest Path	
2.1	Dijkstra	1
2.2	Bellman-Ford	1
2.3	Floyd-Warshall	2
3	DP	
3.1	0-1 Knapsack	2
3.2	Unbounded Knapsack	2
3.3	SOS DP	2
3.4	O(lgn) LIS	2
3.5	O(lgn) LCS	2
3.6	Counting DP	2
3.7	Probabilistic DP	3
3.8	Probabilistic DP (機率與期望值 DP)	3
3.9	Bitmask DP	3
4	Graph	
4.1	Euler Path	4
4.2	Hamiltonian path	4
4.3	Kth-Shortest Path	4
4.4	次小生成樹	5
5	Data Structure	
5.1	Segment Tree	6
5.2	Binary Indexed Tree	7
5.3	DSU	7
6	Others	
6.1	Discretization	7
6.2	Binary Splitting	7
6.3	3D LIS	7
6.4	倍增	8
6.5	LCA	8
6.6	Area of Rectangles	8

1 Math

1.1 Fpow

```
int fpow(int x, int n, int mod) {
    int res = 1;
    while (n) {
        if (n & 1) res = res * x % mod;
        x = x * x % mod;
        n >>= 1;
    }
    return res;
}
```

1.2 Combination

```
#define MXN 1'000'005
#define N 1'000'000
#define ll long long

ll fac[MXN], inv[MXN];
fac[0] = 1; // 0! = 1
for(int i = 1; i <= N; i++) fac[i] = fac[i-1] * i % mod;
inv[N] = fpow(fac[N], mod-2); // 快速幂
for(int i = N-1; i >= 0; i--) inv[i] = inv[i+1] * (i+1) % mod;

ll C(int n, int k) {
    return fac[n] * inv[k] % mod * inv[n-k] % mod;
}
```

1.3 Extgcd

```
#define ll long long
int extgcd(int a, int b, int &x, int &y) {
    if(b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int GCD=extgcd(b, a%b, y, x);
    y -= a / b * x;
    return GCD;
}

int main(){
    int a, b, x, y;
    cin >> a >> b;
    int GCD = extgcd(a, b, x, y);
}
```

1.4 質數

```
/* 12721, 13331, 14341, 75577, 123457, 222557, 556679
* 999983, 1097774749, 1076767633, 100102021, 999997771
* 1001010013, 1000512343, 987654361, 999991231
* 999888733, 98789101, 987777733, 999991921, 1010101333
* 1010102101, 1000000000039, 100000000000037
* 2305843009213693951, 4611686018427387847
* 9223372036854775783, 18446744073709551557 */
```

1.5 模反元素

```
long long inv(long long a, long long m) {
    long long x, y;
    long long d=exgcd(a, m, x, y);
    if(d==1) return (x+m)%m;
    else return -1; //-1為無解
}
```

2 Shortest Path

2.1 Dijkstra

```
#define pii pair<int,int>
#define INF 1e18
vector<pii>vec[N];
void dijkstra(int s, int t) { //起點 · 終點
    int dis[N];
    for(int i=0; i<N; i++) {
        dis[i]=INF; //值要設為比可能的最短路徑權重還要大的值
    }
    dis[s]=0;
    priority_queue<pii, vector<pii>, greater<pii>>pq;
    pq.push({dis[s], s});
    while(pq.empty()==0) {
        int u=pq.top().second;
        pq.pop();
        if(vis[u]) continue;
        vis[u]=1;
        for(auto [v,w]:vec[u]) {
            if(dis[u]+w<dis[v]) {
                dis[v]=dis[u]+w;
                pq.push({dis[v], v});
            }
        }
    }
}
```

2.2 Bellman-Ford

```
struct edge {
    int v, w;
};
int s, n; // 起點 點數量
```

```
vector<edge> E[MXN];
vector<int> dis(MXN, INF), cnt(MXN), inq(MXN);
// cnt記錄最短路經過幾條邊
// inq記錄節點是否在queue裡面
queue<int> Q;
dis[s] = 0; Q.push(s); inq[s] = 1;
while (Q.size()) {
    int u = Q.front(); Q.pop();
    inq[u] = 0;
    for (auto [v, w] : E[u]) {
        if (dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            cnt[v] = cnt[u] + 1;
            if (cnt[v] >= n) {有負環}
            if (!inq[v]) Q.push(v), inq[v] = 1;
        }
    }
}
```

2.3 Floyd-Warshall

```
for(int k=1;k<=N;k++){//窮舉中繼點k
    for(int i=1;i<=N;i++){
        for(int j=1;j<=N;j++){//窮舉點對(i,j)
            dis[i][j]=min(dis[i][j],dis[i][k]+dis[k][j]);
        }
    }
}
```

3 DP

3.1 0-1 Knapsack

```
for(int i = 1; i <= n; ++i){
    for(int w = W; w >= 0; --w)
        dp[w] = max(w - c[i] >= 0 ? dp[w - c[i]] + v[i] : 0, dp[w]);

    // Backtracing
    for(int i = 1; i <= n; ++i){
        for(int w = 0; w <= W; ++w) {
            //dp[i][w] 考慮前 i 個物品且負重上限為 w 的答案。
            int a = w - c[i] >= 0 ? dp[i - 1][w - c[i]] + v[i] : 0;
            int b = dp[i - 1][w];
            if(a > b){
                dp[i][w] = a;
                R[i][w] = w - c[i];
            }else{
                dp[i][w] = b;
                R[i][w] = w;
            }
        }
    }

    int i = n, w = W;
    vector<int> ans;
    while(i){
        if(w != R[i][w]) ans.push_back(i); //不一樣代表有取
        w = R[i][w];
        --i;
    }
}
```

3.2 Unbounded Knapsack

```
for(int w = 0; w <= W; ++w)
    for(int i = 1; i <= n; ++i)
        dp[w] = max(dp[w], w - c[i] >= 0 ? dp[w - c[i]] + v[i] : 0);
```

3.3 SOS DP

目標是解決求給定集合的所有子集和：

$$F(S) = \sum_{x \subseteq S} A(x)$$

直接窮舉子集複雜度為 $O(3^n)$ ，使用 SOS DP 則可優化至 $O(n \cdot 2^n)$ 。

高維前綴和概念 (High-Dimensional Prefix Sum)

SOS DP 的本質其實就是高維度的前綴和推廣：

1. **一維前綴和**：沿著單一方向累加 $F[x] += F[x-1]$ 。
2. **二維前綴和**：先固定 Y 對 X 做一維前綴和 ($F[x][y] += F[x-1][y]$)，再固定 X 對 Y 做一維前綴和 ($F[x][y] += F[x][y-1]$)。幾何上就是把一條條的前綴和 (線) 疊加成一個矩形 (面)。
3. **N 維前綴和 (SOS DP)**：在 $bitmask$ 中，共有 n 個維度，每個維度的大小只有 0 和 1。我們只需依序窮舉每一維 i ，若當前狀態 s 在該維度是 1 (即 $s \& (1 \ll i)$ 為真)，就將長度較短的狀態 (即該維度為 0 的狀態 $s \oplus (1 \ll i)$) 累加到 $F[s]$ 中。

```
for(int i = 0; i < (1 << N); ++i)
    F[i] = A[i];
for(int i = 0; i < N; ++i)
    for(int mask = 0; mask < (1 << N); ++mask)
        if(mask & (1 << i))
            F[mask] += F[mask ^ (1 << i)];
//窮舉子集
for(int s = mask; s; s = (s - 1) & mask;)
```

3.4 O(lgn) LIS

```
#define all(v) (v).begin(), (v).end()

vector<int> dp;
for(int i = 0; i < n; ++i){
    auto it = lower_bound(all(dp), A[i]);
    if(it == dp.end()){
        dp.push_back(A[i]);
    }else{
        *it = A[i];
    }
}
cout << dp.size() << '\n';
```

3.5 O(lgn) LCS

```
int LCS(string& s1, string& s2) {
    vector<int> p[128]; // 假設字元範圍為 0 ~ 127
    for (int i = 0; i < s2.size(); ++i) p[s2[i]].push_back(i);
    vector<int> v;
    v.push_back(-1);

    for (int i = 0; i < s1.size(); ++i)
        for (int j = p[s1[i]].size() - 1; j >= 0; --j) {
            int n = p[s1[i]][j];

            if (n > v.back())
                v.push_back(n);
            else
                *lower_bound(v.begin(), v.end(), n) = n;
        }
    return v.size() - 1;
};
```

3.6 Counting DP

用於計算方法數的 DP，通常可以配合排列組合與排容原理。

障礙物棋盤走路計數問題

給定 $n \times m$ 棋盤與 N 個障礙物，只能向下或右走，求起點到終點且不經過障礙物的方法數。利用總和扣除不合路徑的方法：

$$dp[D] = cnt(O, D) - \sum_{\{k\}} cnt(k, D) \cdot dp[k]$$

轉移順序：將點先按 X 排序，再按 Y 排序，確保 partial-ordering 正確。

```
vector<pair<int, int>> A(N);
sort(A.begin(), A.end());

// Insert start point at index 0 and end point at back
A.insert(A.begin(), pair<int, int>{1, 1});
A.push_back(pair<int, int>{n, m});

vector<int> dp(N + 5);

for(int i = 1; i <= N + 1; ++i) {
    // Calculate total valid permutations without obstacles
```

```

dp[i] = cnt(0, i);
auto [x, y] = A[i];

// Subtract combinations that pass through other
// obstacles
for(int j = 1; j < i; ++j) {
    auto [xx, yy] = A[j];

    // Ensure obstacle j is strictly within the path to
    // obstacle i
    if(xx <= x && yy <= y) {
        dp[i] -= cnt(j, i) * dp[j];
    }
}
}
cout << dp[N+1];

```

3.7 Probabilistic DP

3.8 Probabilistic DP (機率與期望值 DP)

核心為利用 DP 狀態轉移模擬全機率公式與期望值的線性性質。

一、兩大轉移方向

- 求機率 (通常順推 Forward): 定義 $dp[i]$ 為從起點抵達狀態 i 的機率。初始 $dp[start] = 1$ 。
轉移方程: $dp[next] += dp[cur] \times P(cur \rightarrow next)$
- 求期望值 (通常逆推 Backward): 定義 $E[i]$ 為從狀態 i 至終點的期望步數或花費。初始 $E[end] = 0$ 。
轉移方程: $E[cur] = \sum P(cur \rightarrow next) \times (E[next] + cost)$
優勢: 逆推可自動過濾無法抵達終點的無效狀態, 確保計算收斂。

二、經典模型與狀態設計

- 擲骰子模型 (古典機率與背包): 處理獨立事件的多次操作 (屬有向無環圖 DAG 轉移)。
精度細節: 迴圈內避免頻繁除法。應先累加前置狀態的總和, 最後再統一除以分支數。
- 抓老鼠模型 (雙方博弈與條件機率): 處理輪流操作且目標隨機減少的問題。轉移時需窮舉當前所有可能事件分支 (例如: 主角直接贏、對手抽中干擾且引發不同的後續變化)。
實作細節: 必須明確定義極端邊界條件 (如只有目標物機率為 1, 只有干擾物機率為 0)。

三、實戰 Checklist

- 圖中是否有環? (Cyclic Transitions)
若狀態轉移成環 (如有一定機率停留原地), 常規 DP 會死循環。此時需將轉移式視為線性聯立方程式, 建立矩陣並使用高斯消去法 (Gaussian Elimination) 求解。
- 浮點數型別:
只要牽涉機率與期望值, 一律使用 double 或 long double。
- 期望值的線性性質:
利用 $E[A + B] = E[A] + E[B]$ 。處理多個獨立物件時, 無需將其壓縮成單一高維狀態, 可分拆計算各自期望值後相加, 大幅降低時間複雜度。

骰子問題 (Dice Probabilities)

擲 n 次六面骰子, 所有結果相加落在 $[a, b]$ 的機率。

```

// dp[i][j] represents the probability of reaching sum
// j after i rolls
double dp[N+1][6*N+1];

// Initialize the first roll probability
for(int i = 1; i <= 6; ++i) {
    dp[1][i] = 1.0 / 6.0;
}

for(int i = 2; i <= n; ++i) {
    // Max sum after n rolls is 6*n (assuming n <= 100,
    // max 600)
    for(int w = 1; w <= 600; ++w) {
        // Iterate through all 6 dice faces
        for(int j = 1; j <= 6; ++j) {
            if(w >= j) {
                dp[i][w] += dp[i-1][w-j];
            }
        }
    }
}

```

```

// Divide once per state to prevent precision
// errors
dp[i][w] /= 6.0;
}
}

double ans = 0.0;
for(int i = a; i <= b; ++i) {
    ans += dp[n][i];
}

```

抓老鼠期望值問題 (Mice Probability)

袋中有 w 隻白鼠、 b 隻黑鼠。雙方輪流抓, 抓到白鼠贏, 每次抓完黑鼠有機率跑出一隻。

```

using ld = long double;
ld dp[1005][1005];

// Initialize: when only white mice remain, princess
// wins 100%
for (int i = 1; i <= w; i++) dp[i][0] = 1;
// Initialize: when only black mice remain, princess
// wins 0%
for (int i = 1; i <= b; i++) dp[0][i] = 0;

for (int i = 1; i <= w; i++) {
    for (int j = 1; j <= b; j++) {
        auto &cur = dp[i][j];

        // Case 1: Princess catches a white mouse
        // immediately
        cur += (ld)i / (i + j);

        // Probability that both princess and dragon catch
        // a black mouse
        ld x = (ld)j / (i + j) * (j - 1) / (i + j - 1);

        // Case 2: A black mouse escapes after they both
        // catch black mice
        if (j >= 3) {
            cur += x * (j - 2) / (i + j - 2) * dp[i][j-3];
        }

        // Case 3: A white mouse escapes after they both
        // catch black mice
        if (j >= 2) {
            cur += x * i / (i + j - 2) * dp[i-1][j-2];
        }
    }
}

```

3.9 Bitmask DP

位元壓縮 DP, 泛指狀態是由二進制表示的 DP。通常可以表示集合, 0 表示不取, 1 表示取。假設有 n 個東西要考慮, 使用長度為 n 的 bitstring 表示狀態, 複雜度通常為 $O(2^n)$ 。看到題目範圍 $n \leq 20$ 即可考慮此解法。

搭電梯問題 (Elevator Problem)

n 個人排隊, 每個人重 w_i , 要搭一台負重 x 的電梯, 問最少要搭幾趟? 利用 $O(2^n)$ 窮舉最後上電梯的人, 減少了窮舉 $n!$ 排列的時間複雜度。

```

#define pp pair<int,int>
#define ff first
#define ss second
pp dp[(1<<20)]; //2^20
void solve(){
    int n,x;
    cin >> n >> x;
    vector<int> w(n);
    for(auto &c:w) cin >> c;
    //初始化: 電梯最多搭 n 次, 所以開 n+1 當 inf
    for(int s = 0; s < (1<<n); ++s) dp[s].ff = n+1;
    // s 為集合
    dp[0].ss = x, dp[0].ff = 0;
    //沒人上電梯 讓空集合電梯目前的負重是 x
    //所以空集這台電梯就搭不上去, 這樣下一個人上電梯一
    //定會搭到下一班
    for(int s = 1; s < (1<<n); ++s){ //窮舉全部狀態
        for(int i = 0; i < n; ++i){
            if(!(s & (1 << i))) continue;
            // i 為此集合 s 最後上電梯的人 所以必須落在
            // 集合 s 內
            int pv = s ^ (1 << i); // i 沒搭的狀態

```

```

    auto [a,b] = dp[pv];

    if(b + w[i] > x) b = w[i], ++a; // i 搭不上最後一班，得多一班
    else b += w[i]; //搭的上去

    if(a < dp[s].ff) dp[s] = pp(a,b); //次數少先取
    else if(a == dp[s].ff) dp[s].ss = min(dp[s].ss,b); //次數一樣，取最小的負重
  }
}
cout << dp[(1<<n)-1].ff << '\n';
}

```

4 Graph

4.1 Euler Path

```

vector<int> path;
void dfs(int x){
    while(!edge[x].empty()){
        int u = edge[x].back();
        edge[x].pop_back();
        dfs(u);
    }
    path.push_back(x);
}
int main(){
    dfs(st);
    reverse(path.begin(), path.end());
    for(int i: path) cout<<i<<' ';
}

```

4.2 Hamiltonian path

```

// 圖中有 n 個點，詢問是否能走過所有點且每個點僅經過一次。需要記錄經過了哪些點，以及停留在哪個點。
// Initialize: each node can reach itself
for(int i = 0; i < n; ++i) {
    dp[(1<<i)][i] = true;
}

// Iterate through all subsets
for(int s = 1; s < (1<<n); ++s) {
    for(int j = 0; j < n; ++j) {
        // Continue if vertex j is not in state s
        if(s & (1<<j)) continue;

        // Iterate all endpoints in state s
        for(int i = 0; i < n; ++i) {
            // s must pass through i
            if(!(s & (1<<i))) continue;

            // If there is a directed edge from i to j
            if(edge[i][j]) {
                dp[s | (1<<j)][j] |= dp[s][i];
            }
        }
    }
}

bool ans = false;
for(int i = 0; i < n; ++i) ans |= dp[(1<<n)-1][i];

```

4.3 Kth-Shortest Path

```

// time: O(|E| \lg |E| + |V| \lg |V| + K)
// memory: O(|E| \lg |E| + |V|)
struct KSP{ // 1-base
    struct nd{
        int u, v; ll d;
        nd(int ui = 0, int vi = 0, ll di = INF)
        { u = ui; v = vi; d = di; }
    };
};

```

```

struct heap{
    nd* edge; int dep; heap* chd[4];
};
static int cmp(heap* a, heap* b)
{ return a->edge->d > b->edge->d; }
struct node{
    int v; ll d; heap* H; nd* E;
    node(){
        node(ll _d, int _v, nd* _E)
        { d = _d; v = _v; E = _E; }
        node(heap* _H, ll _d)
        { H = _H; d = _d; }
        friend bool operator<(node a, node b)
        { return a.d > b.d; }
    };
};
int n, k, s, t;
ll dst[ N ];
nd *nxt[ N ];
vector<nd*> g[ N ], rg[ N ];
heap *nullNd, *head[ N ];
void init( int _n, int _k, int _s, int _t ){
    n = _n; k = _k; s = _s; t = _t;
    for( int i = 1; i <= n; i ++ ){
        g[ i ].clear(); rg[ i ].clear();
        nxt[ i ] = NULL; head[ i ] = NULL;
        dst[ i ] = -1;
    }
}
void addEdge( int ui, int vi, ll di ){
    nd* e = new nd(ui, vi, di);
    g[ ui ].push_back( e );
    rg[ vi ].push_back( e );
}
queue<int> dfsQ;
void dijkstra(){
    while(dfsQ.size()) dfsQ.pop();
    priority_queue<node> Q;
    Q.push(node(0, t, NULL));
    while (!Q.empty()){
        node p = Q.top(); Q.pop();
        if(dst[p.v] != -1) continue;
        dst[p.v] = p.d;
        nxt[p.v] = p.E;
        dfsQ.push(p.v);
        for(auto e: rg[p.v])
            Q.push(node(p.d + e->d, e->u, e));
    }
}
heap* merge(heap* curNd, heap* newNd){
    if(curNd == nullNd) return newNd;
    heap* root = new heap;
    memcpy(root, curNd, sizeof(heap));
    if(newNd->edge->d < curNd->edge->d){
        root->edge = newNd->edge;
        root->chd[2] = newNd->chd[2];
        root->chd[3] = newNd->chd[3];
        newNd->edge = curNd->edge;
        newNd->chd[2] = curNd->chd[2];
        newNd->chd[3] = curNd->chd[3];
    }
    if(root->chd[0]->dep < root->chd[1]->dep)
        root->chd[0] = merge(root->chd[0], newNd);
    else
        root->chd[1] = merge(root->chd[1], newNd);
    root->dep = max(root->chd[0]->dep, root->chd[1]->dep) + 1;
    return root;
}
vector<heap*> V;
void build(){
    nullNd = new heap;
    nullNd->dep = 0;
    nullNd->edge = new nd;
    fill(nullNd->chd, nullNd->chd+4, nullNd);
    while(not dfsQ.empty()){
        int u = dfsQ.front(); dfsQ.pop();
        if(!nxt[u]) head[u] = nullNd;
        else head[u] = head[nxt[u]->v];
        V.clear();
        for( auto& e : g[u] ){
            int v = e->v;
            if( dst[v] == -1 ) continue;
            e->d += dst[v] - dst[u];
            if( nxt[u] != e ){

```

```

        heap* p = new heap;
        fill(p->chd, p->chd+4, nullNd);
        p->dep = 1;
        p->edge = e;
        V.push_back(p);
    } }
    if(V.empty()) continue;
    make_heap(V.begin(), V.end(), cmp);
#define L(X) ((X<<1)+1)
#define R(X) ((X<<1)+2)
    for( size_t i = 0 ; i < V.size() ; i ++ ){
        if(L(i) < V.size()) V[i]->chd[2] = V[L(i)];
        else V[i]->chd[2]=nullNd;
        if(R(i) < V.size()) V[i]->chd[3] = V[R(i)];
        else V[i]->chd[3]=nullNd;
    }
    head[u] = merge(head[u], V.front());
} }
vector<ll> ans;
void first_K(){
    ans.clear();
    priority_queue<node> Q;
    if( dst[ s ] == -1 ) return;
    ans.push_back( dst[ s ] );
    if( head[s] != nullNd )
        Q.push(node(head[s], dst[s]+head[s]->edge->d));
    for( int _ = 1 ; _ < k and not Q.empty() ; _ ++ ){
        node p = Q.top(), q; Q.pop();
        ans.push_back( p.d );
        if(head[ p.H->edge->v ] != nullNd){
            q.H = head[ p.H->edge->v ];
            q.d = p.d + q.H->edge->d;
            Q.push(q);
        }
        for( int i = 0 ; i < 4 ; i ++ )
            if( p.H->chd[ i ] != nullNd ){
                q.H = p.H->chd[ i ];
                q.d = p.d - p.H->edge->d + p.H->chd[ i ]->
                    edge->d;
                Q.push( q );
            }
    }
    void solve(){ // ans[i] stores the i-th shortest path
        dijkstra();
        build();
        first_K(); // ans.size() might less than k
    } } solver;

```

4.4 次小生成樹

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

const long long INF = 1e18;
const int MAXN = 100005;
const int LOG = 19;

struct Edge {
    int u, v, w;
    bool in_mst;
    // Sort edges by weight in ascending order
    bool operator<(const Edge& other) const {
        return w < other.w;
    }
};

// DSU for Kruskal's Algorithm
struct DSU {
    vector<int> fa;
    DSU(int n) {
        fa.resize(n + 1);
        for (int i = 1; i <= n; ++i) fa[i] = i;
    }
    int find(int x) {
        return fa[x] == x ? x : fa[x] = find(fa[x]);
    }
    bool merge(int x, int y) {
        int fx = find(x), fy = find(y);

```

```

        if (fx == fy) return false;
        fa[fy] = fx;
        return true;
    }
};

int n, m;
vector<Edge> edges;
vector<pair<int, int>> adj[MAXN]; // Tree formed by MST edges

// Binary Lifting Tables
int depth[MAXN];
int up[MAXN][LOG];
long long max1_edge[MAXN][LOG]; // max1_edge: The
    maximum edge weight on the path
long long max2_edge[MAXN][LOG]; // max2_edge: The
    STRICTLY second maximum edge weight

// Helper function to update the top 2 distinct
    maximums from a set of values
void update_max(long long& m1, long long& m2, long long
    val) {
    if (val > m1) {
        m2 = m1;
        m1 = val;
    } else if (val < m1 && val > m2) {
        m2 = val;
    }
}

// DFS to initialize depth and 0-th ancestor (parent)
void dfs(int u, int p, int w, int d) {
    depth[u] = d;
    up[u][0] = p;
    max1_edge[u][0] = w;
    max2_edge[u][0] = -INF;

    // Build the binary lifting tables
    for (int i = 1; i < LOG; ++i) {
        int ancestor = up[u][i - 1];
        up[u][i] = up[ancestor][i - 1];

        long long m1 = -INF, m2 = -INF;
        // Merge the top 2 max edges from the lower half
            and upper half
        update_max(m1, m2, max1_edge[u][i - 1]);
        update_max(m1, m2, max2_edge[u][i - 1]);
        update_max(m1, m2, max1_edge[ancestor][i - 1]);
        update_max(m1, m2, max2_edge[ancestor][i - 1]);

        max1_edge[u][i] = m1;
        max2_edge[u][i] = m2;
    }

    // Traverse children
    for (auto& edge : adj[u]) {
        int v = edge.first;
        int weight = edge.second;
        if (v != p) {
            dfs(v, u, weight, d + 1);
        }
    }
}

// Function to query the max and strict second max
    edges on the path between u and v
pair<long long, long long> query_path(int u, int v) {
    long long res_m1 = -INF, res_m2 = -INF;

    // Ensure u is deeper than v
    if (depth[u] < depth[v]) swap(u, v);

    // Equalize depths
    for (int i = LOG - 1; i >= 0; --i) {
        if (depth[u] - (1 << i) >= depth[v]) {
            update_max(res_m1, res_m2, max1_edge[u][i]);
            update_max(res_m1, res_m2, max2_edge[u][i]);
            u = up[u][i];
        }
    }
}

```

```

if (u == v) return {res_m1, res_m2};

// Climb together to find LCA
for (int i = LOG - 1; i >= 0; --i) {
    if (up[u][i] != up[v][i]) {
        update_max(res_m1, res_m2, max1_edge[u][i]);
        update_max(res_m1, res_m2, max2_edge[u][i]);
        update_max(res_m1, res_m2, max1_edge[v][i]);
        update_max(res_m1, res_m2, max2_edge[v][i]);
        u = up[u][i];
        v = up[v][i];
    }
}

// Process the last step to LCA
update_max(res_m1, res_m2, max1_edge[u][0]);
update_max(res_m1, res_m2, max2_edge[v][0]);

return {res_m1, res_m2};
}

void solve() {
    // 1. Find Minimum Spanning Tree (MST) using Kruskal
    sort(edges.begin(), edges.end());
    DSU dsu(n);
    long long mst_weight = 0;
    int edge_cnt = 0;

    for (auto& e : edges) {
        e.in_mst = false;
        if (dsu.merge(e.u, e.v)) {
            e.in_mst = true;
            mst_weight += e.w;
            edge_cnt++;
            adj[e.u].push_back({e.v, e.w});
            adj[e.v].push_back({e.u, e.w});
        }
    }

    // Check if MST is possible
    if (edge_cnt != n - 1) {
        // cout << "No Spanning Tree" << "\n";
        return;
    }

    // 2. Precompute LCA and max edges tables
    dfs(1, 1, 0, 0);

    // 3. Find the Second MST
    long long strictly_second_mst = INF;
    long long non_strictly_second_mst = INF;

    for (const auto& e : edges) {
        if (e.in_mst) continue; // Only consider edges NOT
                                // in the MST

        pair<long long, long long> path_max = query_path(e.
            u, e.v);
        long long m1 = path_max.first;
        long long m2 = path_max.second;

        // DIFFERENCE 1: Non-Strictly Second MST
        // We can ALWAYS replace the max edge (m1) because
        // the new edge e.w is guaranteed to be >= m1
        non_strictly_second_mst = min(
            non_strictly_second_mst, mst_weight - m1 + e.w
        );

        // DIFFERENCE 2: Strictly Second MST
        // We must ensure the new total weight is STRICTLY
        // greater than the original MST weight
        if (e.w > m1) {
            strictly_second_mst = min(strictly_second_mst,
                mst_weight - m1 + e.w);
        } else if (e.w == m1 && m2 != -INF) {
            strictly_second_mst = min(strictly_second_mst,
                mst_weight - m2 + e.w);
        }
    }

    // Output the results based on your needs
    // cout << "MST: " << mst_weight << "\n";

```

```

// cout << "Non-Strict: " << non_strictly_second_mst
// << "\n";
// cout << "Strict: " << strictly_second_mst << "\n";
}

```

5 Data Structure

5.1 Segment Tree

```

// !!!注意build()時初始化用的陣列也是1-base
#define cl(x) (x*2)
#define cr(x) (x*2+1)

struct segmentTree {
    int n;
    vector<int> seg, tag, cov;
    segmentTree(int _n) : n(_n) {
        seg=tag=cov=vector<int>(n*4,0);
    }
    void push(int i, int L, int R) {
        if (cov[i]) {
            seg[i]=cov[i]*(R-L+1);
            if (L < R) {
                cov[cl(i)]=cov[cr(i)]=cov[i];
                tag[cl(i)]=tag[cr(i)]=0;
            }
            cov[i]=0;
        }
        if (tag[i]) {
            seg[i]+=tag[i]*(R-L+1);
            if (L < R) {
                tag[cl(i)]=tag[i];
                tag[cr(i)]=tag[i];
            }
            tag[i]=0;
        }
    }
    void pull(int i, int L, int R) {
        if (L >= R) return;
        int mid=(L+R)>>1;
        push(cl(i),L,mid);
        push(cr(i),mid+1,R);
        seg[i]=seg[cl(i)]+seg[cr(i)];
    }
    void build(vector<int>& arr, int i=1, int L=1, int R=-1) {
        if (R == -1) R=n;
        if (L == R) {
            seg[i]=arr[L];
            return;
        }
        int mid=(L+R)>>1;
        build(arr,cl(i),L,mid);
        build(arr,cr(i),mid+1,R);
        pull(i,L,R);
    }
    int query(int rL, int rR, int i=1, int L=1, int R=-1) {
        if (R == -1) R=n;
        push(i,L,R);
        if (rL <= L && R <= rR) return seg[i];
        int mid=(L+R)>>1, ret=0;
        if (rL <= mid) ret+=query(rL,rR,cl(i),L,mid);
        if (mid < rR) ret+=query(rL,rR,cr(i),mid+1,R);
        return ret;
    }
    void update(int rL, int rR, int val, int i=1, int L=1, int R=-1) {
        if (R == -1) R=n;
        push(i,L,R);
        if (rL <= L && R <= rR) {
            tag[i]=val;
            return;
        }
        int mid=(L+R)>>1;
        if (rL <= mid) update(rL,rR,val,cl(i),L,mid);
        if (mid < rR) update(rL,rR,val,cr(i),mid+1,R);
        pull(i,L,R);
    }

```

```

}
void cover( int rL, int rR, int val, int i=1, int L
           =1, int R=-1 ) {
    if( R == -1 ) R=n;
    push(i,L,R);
    if( rL <= L && R <= rR ) {
        cov[i]=val;
        return;
    }
    int mid=(L+R)>>1;
    if( rL <= mid ) cover(rL,rR,val,cl(i),L,mid);
    if( mid < rR ) cover(rL,rR,val,cr(i),mid+1,R);
    pull(i,L,R);
}
};

/* Test Case:
4
1 2 3 4
5
2 1 3
1 1 3 1
2 1 3
1 1 4 1
2 1 4
*/

```

5.2 Binary Indexed Tree

```

#define lowbit(x) (x&-x)
struct BIT {
    int n;
    vector<int> bit;
    BIT(int _n):n(_n), bit(_n+1), C(_n+1) {}
    void update(int x, int val) {
        for(; x <= n; x += lowbit(x)) bit[x] += val;
    }
    void update(int L, int R, int val) {
        update(L, val), update(R+1, -val);
    }
    int query(int x) {
        int res = 0;
        for(; x; x -= lowbit(x)) res += bit[x];
        return res;
    }
    int query(int L, int R) {
        return query(R) - query(L-1);
    }
    int getmax(int l, int r) {
        int ans = 0;
        while(l <= r) {
            ans = max(ans, bit[r--]);
            for (; l <= r - lowbit(r); r -= lowbit(r)) ans
                = max(ans, C[r]);
        }
        return ans;
    }
    int kth(int k) {
        int sum = 0, x = 0;
        for (int i = __lg(n); ~i; i--) {
            x += 1 << i;
            if (x >= n || sum + bit[x] >= k) x -= 1 << i;
            else sum += bit[x];
        }
        return x + 1;
    }
};

```

5.3 DSU

```

struct DSU {
    int n;
    vector<int> fa, sz;
    vector<tuple<int, int, int, int>> ver;
    DSU(int _n): n(_n), fa(n), sz(n, 1) {
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) {

```

```

        return fa[x] == x ? x : find(fa[x]);
    }
    void merge(int x, int y) {
        x = find(x), y = find(y);
        if(sz[x] < sz[y]) swap(x, y);
        ver.push_back({x, sz[x], y, fa[y]});
        if(x == y) return;
        sz[x] += sz[y];
        fa[y] = x;
    }
    void undo() {
        if(ver.empty()) return;
        auto [x, szx, y, fy] = ver.back();
        ver.pop_back();
        sz[x] = szx;
        fa[y] = fy;
    }
};

```

6 Others

6.1 Discretization

```

//vector<int> arr;
vector<int> tmp(arr); //將arr複製到tmp
sort(tmp.begin(), tmp.end());
tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end());
for (int i = 0; i < n; i++)
    arr[i] = lower_bound(tmp.begin(), tmp.end(), arr[i])
        - tmp.begin();

```

6.2 Binary Splitting

```

//目前物品最多拿 k 個
int s = 0;
vector<pair<int,int>> A; // A 是儲存所有捆
for(int r = 1; r * 2 <= k; r *= 2){
    s += r;
    A.push_back({r*c[i],r*v[i]}); // r 個一捆
}
A.push_back({(k-s)*c[i],(k-s)*v[i]}); // 最後一捆

```

6.3 3D LIS

```

#define lowbit(x) (x&-x)
const int MAXN=1e5+5;
struct BIT {
    int n;
    vector<int> bit;
    BIT( int _n ):n(_n), bit(_n+1,0) {}
    int query( int x ) {
        int res=0;
        for(; x > 0 ; x-=lowbit(x) ) res=max(res,bit[x]);
        return res;
    }
    void update( int x, int val ) {
        for(; x <= n ; x+=lowbit(x) ) {
            if( val < 0 ) bit[x]=0;
            else bit[x]=max(bit[x],val);
        }
    }
}bt(MAXN);
struct triple {
    int u, v, w, ans, cnt;
    bool operator<( triple b ) { return u<b.u; }
};
bool cmp( triple a, triple b ) {return a.v<b.v;}
void cdq( int L, int R, vector<triple>& arr ) {
    if( R-L <= 1 ) return;
    int mid=L+R>>1;
    cdq(L,mid,arr);
    sort(arr.begin()+L,arr.begin()+mid,cmp);
    sort(arr.begin()+mid,arr.begin()+R,cmp);
    for( int i=L, j=mid ; i < mid || j < R ; ) {

```



```

    for( i < mid && ( j >= R || arr[i].v < arr[j].v )
        ; i++ ) bt.update(arr[i].w, arr[i].ans);
    if( j < R ) {
        arr[j].ans = max(bt.query(arr[j].w-1)+1, arr[j].ans);
        j++;
    }
}
for( int i=L ; i < mid ; i++ ) bt.update(arr[i].w, -1);
sort(arr.begin()+L, arr.begin()+R);
cdq(mid, R, arr);
}
signed main()
{
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    ;
    int n, res=0;
    cin>>n;
    vector<int> ls;
    vector<triple> arr;
    for( int i=0 ; i < n ; i++ ) {
        int a, b;
        cin>>a>>b;
        arr.push_back({i, a, b, 1, 1}); // 第一維, 第二維, 第三維,
        // 答案, 數量
        ls.push_back(b);
    }
    sort(ls.begin(), ls.end());
    ls.resize(unique(ls.begin(), ls.end())-ls.begin());
    for( auto &t : arr ) t.w = lower_bound(ls.begin(), ls.
        end(), t.w)-ls.begin()+1;
    n = arr.size();
    cdq(0, n, arr);
    for( int i=0 ; i < n ; i++ ) res = max(res, arr[i].ans);
    cout<<res<<'\n';
}

```

6.4 倍增

```

int jump[N][logN]; // n個點 最多跳 logn 步
void init(int now, int nxt) { // now 走 1 步 可以走到 nxt
    jump[now][0] = nxt;
}
int jump[N][logN];
for(int i = 1; i <= log2(N); i++) { // 從跳 2^1 次枚舉到
    跳 2^logn 次
    for(int now = 1; now <= N; now++) {
        jump[now][i] = jump[jump[now][i-1]][i-1];
    }
}
int query(int x, int k) {
    int i = log2(k);
    while(k) {
        if((1<<i) <= k) {
            x = jump[x][i];
            k -= (1<<i);
        }
        i--;
    }
    return x; // 最後跳到的地方就是答案
}

// 樹上倍增
int anc[N][logN];
void dfs(int now, int fa) { // 現在的點、父節點
    anc[now][0] = fa; // now 的 0 倍祖先是 fa
    for(auto i : g[now]) {
        if(i == fa) continue;
        dfs(i, now);
    }
}

```

6.5 LCA

```

int getlca(int x, int y) {

```

```

    if (is_ancestor(x, y)) return x; // 如果 u 為 v
    的祖先則 lca 為 u
    if (is_ancestor(y, x)) return y; // 如果 v 為 u
    的祖先則 lca 為 u
    for (int i = logN; i >= 0; i--) { // 判斷 2^logN,
        2^(logN-1), ..., 2^1, 2^0 倍祖先
        if (!is_ancestor(anc[x][i], y)) // 如果 2^i 倍祖先
            不是 v 的祖先
            x = anc[x][i]; // 則往上移動
    }
    return anc[x][0]; // 回傳此點的父節點即為答案
}

```

6.6 Area of Rectangles

// 二維平面上有 n 個長方形，問你這些長方形的聯集面積。
// 而長方形給你左下和右上兩個端點

```

struct AreaofRectangles {
#define cl(x) (x<<1)
#define cr(x) (x<<1|1)
    int n, id, sid;
    pair<int, int> tree[MXN<<3]; // count, area
    vector<int> ind;
    tuple<int, int, int, int> scan[MXN<<1];
    void point(int i, int l, int r) {
        if(tree[i].first) tree[i].second = ind[r+1] - ind[
            l]; // 當前區間有被做加值
        else if(l != r) { // 沒有就直接拿兩個子節點的第二個數
            字
            int mid = (l+r)>>1;
            tree[i].second = tree[cl(i)].second + tree[cr(i)
                ].second;
        }
        else tree[i].second = 0;
    }
    void upd(int i, int l, int r, int ql, int qr, int v) {
        if(ql <= l && r <= qr) {
            tree[i].first += v;
            point(i, l, r); return;
        }
        int mid = (l+r)>>1;
        if(ql <= mid) upd(cl(i), l, mid, ql, qr, v);
        if(qr > mid) upd(cr(i), mid+1, r, ql, qr, v);
        point(i, l, r);
    }
    void init(int _n) {
        n = _n; id = sid = 0;
        ind.clear(); ind.resize(n<<1);
        fill(tree, tree+(n<<2), make_pair(0, 0));
    }
    void addRectangle(int lx, int ly, int rx, int ry) {
        ind[id++] = lx; ind[id++] = rx;
        scan[sid++] = make_tuple(ly, l, lx, rx);
        scan[sid++] = make_tuple(ry, -1, lx, rx);
    }
    int solve() {
        sort(ind.begin(), ind.end());
        ind.resize(unique(ind.begin(), ind.end()) - ind.
            begin()); // 離散化
        sort(scan, scan + sid);
        int area = 0, pre = get<0>(scan[0]);
        for(int i = 0; i < sid; i++) {
            auto [x, v, l, r] = scan[i];
            area += tree[l].second * (x-pre); //
            upd(1, 0, ind.size()-1, lower_bound(ind.begin(),
                ind.end(), l)-ind.begin(), lower_bound(ind.
                begin(), ind.end(), r)-ind.begin()-1, v);
            pre = x;
        }
        return area;
    }
} rect;

```

```

int main() {
    int n;
    cin >> n;
    rect.init(n);
    for (int i = 0; i < n; i++) {
        int lx, ly, rx, ry;

```



```
    cin >> lx >> ly >> rx >> ry;
    rect.addRectangle(lx, ly, rx, ry);
}
cout << rect.solve() << "\n";
return 0;
}
```